Theory Supplement

Topic

Topic 2 - College Login & Security Module

Group Members

Prajeeta Pathare (PRN: 202401040322)
Ojas Wankhede (PRN: 202401040326)
Himanshu Patil (PRN: 202401040328)
Atharva Lokhande (PRN: 202401040278)

1. Validation Fields

- Email
- Username
- Password
- Confirm Password
- OTP

2. Regular Expressions Used

Field	Regular Expression
Email	`^[A-Za-z0-9._%+-]+@mitaoe\.ac\.in$`
Username	`^[A-Za-z]{3,15}$`
Password	`^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$`
OTP	`^\d{6}$`
Confirm Password	Compared logically with the Password field

3. Formal Language Description

3.1 Email

`L_email = { w | w = x@mitaoe.ac.in, x belongs to [A-Za-z0-9._%+-]+ }`

This language is treated as regular for form validation because it is defined using a regular expression over a finite alphabet.

3.2 Username

`L_username = { w | w belongs to [A-Za-z]{3,15} }`

This language contains all strings of length 3 to 15 made only from uppercase and lowercase English letters.

3.3 Password

`L_password = { w | |w| >= 8 and w contains at least one uppercase letter, one lowercase letter, one digit, and one special symbol from @$!%*?& }`

In implementation, this rule is expressed using regex lookaheads for practical validation.

3.4 OTP

`L_otp = { w | w belongs to [0-9]{6} }`

This language contains all strings made of exactly six digits.

3.5 Confirm Password

`L_confirm = { (x, y) | x = y }`

This is a comparison between two inputs, not a standalone single-string regular language in the same way as the other fields. It is handled using program logic instead of pure regex.

4. Automata Class Justification

Field	Automata Class	Justification
Email	Regular language / Regex / DFA possible	Pattern-based finite validation with fixed symbol rules
Username	Regular language / Regex / DFA	Only letters allowed, bounded length
Password	Regular-rule practical validation	Implemented by regex with lookaheads; suitable for software validation
OTP	Regular language / DFA	Exactly six digits, easy finite-state model
Confirm Password	Beyond pure single-field regex validation	Requires equality check with another field value

5. DFA/NFA for Two Fields

5.1 OTP DFA

Regex: `^\d{6}$`

State sequence:

`q0 -> q1 -> q2 -> q3 -> q4 -> q5 -> q6`

- `q0` is the start state
- each digit moves to the next state
- `q6` is the accepting state
- any non-digit or extra symbol goes to reject state

Accepted examples:

- `123456`
- `000001`
- `987654`

Rejected examples:

- `12345`
- `1234567`
- `12A456`

5.2 Username DFA

Regex: `^[A-Za-z]{3,15}$`

State idea:

- `q0` start
- `q1` after first letter
- `q2` after second letter
- `q3` first accepting state
- continue through `q15`
- any digit, space, or symbol goes to reject state
- length more than 15 also goes to reject state

Accepted examples:

- `Amit`
- `Riya`
- `CollegeUser`

Rejected examples:

- `ab`
- `Riya123`
- `user_name`

6. Course Outcome Mapping

CO.1

The project differentiates regex, DFA, and practical logical validation by showing which fields can be modeled directly using regular languages and which cannot.

CO.2

Regex patterns are constructed for each field, and DFA models are described for Username and OTP.

CO.3

The accepted languages are formally described and checked using valid and invalid test strings.

CO.4

The project identifies OTP and Username as regular-language problems, while Confirm Password is marked as requiring cross-field logic beyond a simple regex.

CO.5

The project shows that regex-based validation is decidable and finite-state, while richer semantic checks need additional program logic.

7. Tools Used

- Browser or Live Server for running the form
- FSM Simulator or JFLAP for drawing DFA/NFA diagrams
- Markdown or PDF conversion for documentation

8. Suggested Figures for Final Submission

- Screenshot of the login form
- Screenshot showing valid submission
- Screenshot showing invalid submission
- OTP DFA diagram
- Username DFA diagram

DFA / NFA Diagrams

Topic 2 – College Login & Security Module

Symbol	Meaning
■ (thin border)	Normal / Intermediate state
■ (blue border, filled)	Start state (→ arrow entering)
■ (green double circle)	Accept / Final state
■ (red border, filled)	Dead / Trap state (rejects & loops on Σ)

---

1. OTP Validation — DFA

Regex: `^\d{6}$`

The OTP field accepts **exactly 6 decimal digits**. The DFA has 7 states (q0–q6) in sequence; each digit advances the state by one. q6 is the sole accepting state. Any non-digit character, or input of length ≠ 6, transitions to the dead state **qd**.

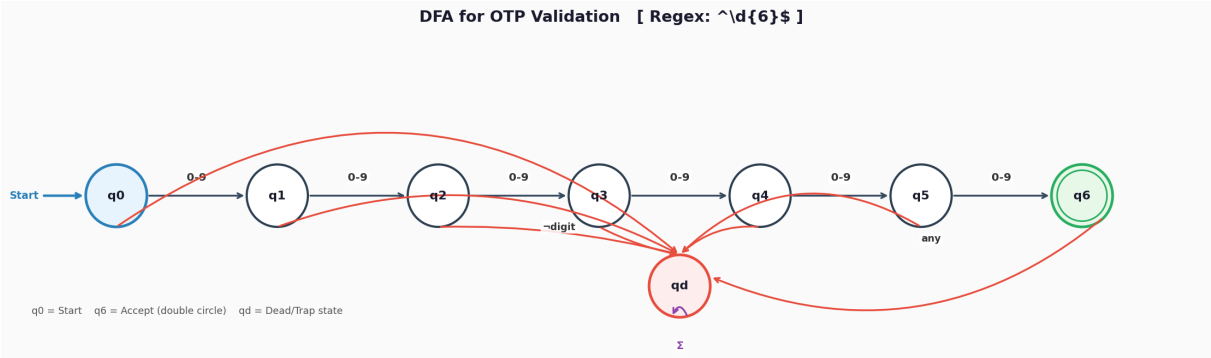

Figure 1 – DFA for OTP | States: q0 (start), q1–q5 (intermediate), q6 (accept), qd (dead)

Test Input	Type	Result
123456	Valid	✓ Accepted
000001	Valid	✓ Accepted
987654	Valid	✓ Accepted
12345	Invalid	✗ Rejected
1234567	Invalid	✗ Rejected
12A456	Invalid	✗ Rejected

---

2. Username Validation — DFA

Regex: `^[A-Za-z]{3,15}$`

The Username field accepts strings of **3 to 15 English letters** only (uppercase or lowercase). The DFA has 17 states: q0 (start), q1–q2 (insufficient length), q3–q15 (accepting — length 3 to 15), and qd (dead). Any non-letter character or a length exceeding 15 leads to qd. *L = [A-Za-z]*.

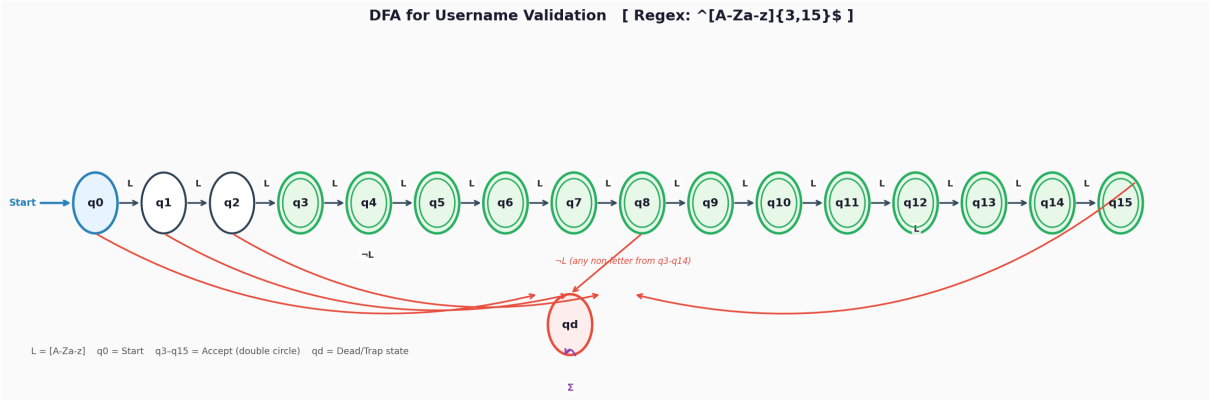

Figure 2 – DFA for Username | q0 start, q3–q15 accept, qd dead

Test Input	Type	Result
Amit	Valid	✓ Accepted
Riya	Valid	✓ Accepted
CollegeUser	Valid	✓ Accepted
ab	Invalid	✗ Rejected
Amit123	Invalid	✗ Rejected
user_name	Invalid	✗ Rejected

3. Email Validation — NFA

Regex: `^[A-Za-z0-9._%+-]+@mitaoe\.ac\.in$`

The Email field requires the fixed domain **@mitaoe.ac.in**. The NFA models the local part (q0→q1 self-loop on valid chars) followed by an exact match of the domain characters one by one through states q2–q14. q14 is the accepting state. Any deviation leads to the dead state **qd**.

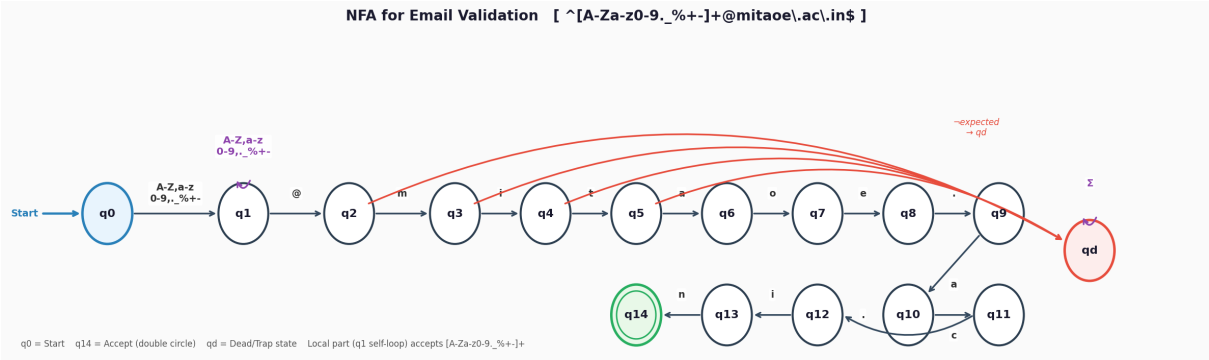

Figure 3 – NFA for Email | q0 start, q14 accept, qd dead

Test Input	Type	Result
student@mitaoe.ac.in	Valid	✓ Accepted
cs2026@mitaoe.ac.in	Valid	✓ Accepted
a.b@mitaoe.ac.in	Valid	✓ Accepted
student@gmail.com	Invalid	✗ Rejected
@mitaoe.ac.in	Invalid	✗ Rejected
user@mitaoe.edu	Invalid	✗ Rejected

4. Password Validation — NFA (Lookahead Model)

Regex: `^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$`

The Password regex uses **lookaheads**, making it naturally an NFA. The NFA reads all characters in a loop (q1), gates on length ≥ 8 at q2, then uses ε-transitions to four parallel checker states (**qL** lowercase, **qU** uppercase, **qD** digit, **qS** special). All four must succeed (AND-semantics) to reach the accepting state **qacc**. Any failure leads to dead state **qd**.

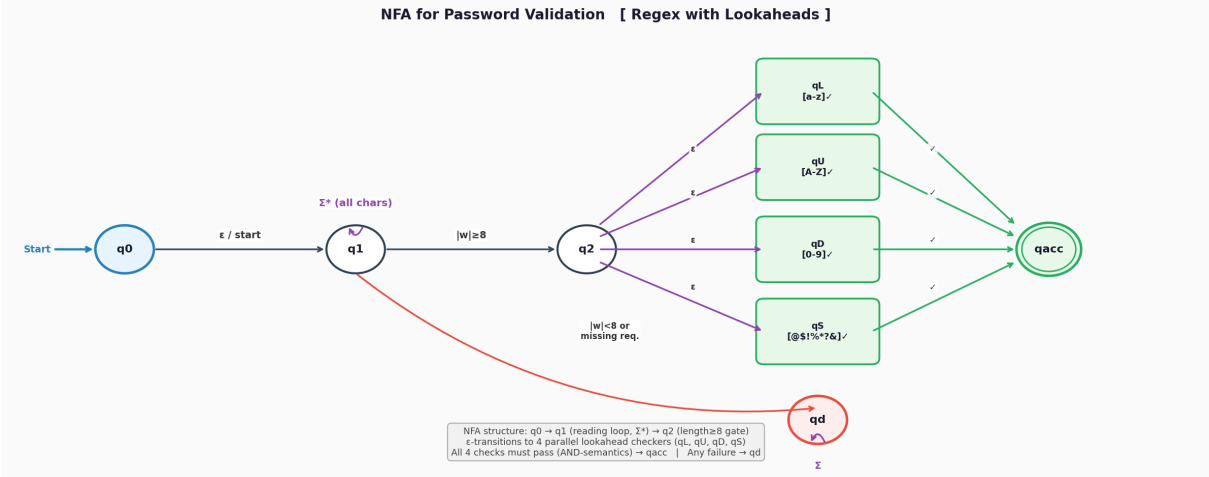

Figure 4 – NFA for Password | q0 start, qacc accept, qd dead

Test Input	Type	Result
Abcd@123	Valid	✓ Accepted
Strong@9A	Valid	✓ Accepted
Secure$45Z	Valid	✓ Accepted
abcd1234	Invalid	✗ Rejected
ABCD@123	Invalid	✗ Rejected
Abcdefgh	Invalid	✗ Rejected

---

5. Confirm Password — Beyond Regular Languages

The Confirm Password field requires **equality with another field**: $L_{confirm} = \{ (x, y) \mid x = y \}$. This is a cross-field check and cannot be expressed as a single-string regular language or modelled by a DFA/NFA in isolation. It is implemented using **program logic** (string equality comparison), not regex. No automaton diagram is applicable.